aktujo

Your documents, kept safe – end to end

Whitepaper 0.1

1. Summary

aktujo takes care of the whole life of a document: **capture** it – scanned, photographed or uploaded –, **improve** it, **read** and **label** it, keep every **version** of it safe, **find** it, **share** it and run it through **workflows** – approved by one person, signed by the next –, every step in an **audit log**. Documents are end-to-end encrypted and stay where their owners want them: on their own devices, synced directly between them, on a server of their own or in the cloud of an organisation.

aktujo is built for every size: from an app on one computer to a cluster run by a Kubernetes operator with PostgreSQL and S3-compatible storage – one edition, the same software. Its core is written in Rust, secure and fast, and is a library as well: aktujo runs as an application of its own, and other applications build it in – in the falkOSS family spezado (accounting), notenwart (sheet music) and kernoid (mail, identity and files).

aktujo is free software published by falkOSS: **everything included** – no paid edition, no licence keys – and **no telemetry**.

2. The problem

Paper and PDFs pile up in every household and every organisation: notices, contracts, policies, payslips, medical letters, warranties, invoices. Handling them takes several tools – a scanner app, a cloud folder, a document manager, a service for signatures, e-mail for approvals – and each of them holds a copy, while nobody can say afterwards who saw, changed, approved or signed what. The documents are among the most personal data there is, yet most of these tools keep them readable to whoever runs the server.

Free document managers exist, but they are servers: they need to be installed and run, they keep documents unencrypted, and taking a picture with a phone or signing a document needs other tools (assessment, **OPEN**: to be verified against paperless-ngx and others before publication).

And every application that wants to work offline, sync between devices and encrypt end to end builds that layer again – in the falkOSS family alone there are three ways of storing files today.

3. Principles

- **Your documents, your choice of place.** Local-first: aktujo works on one device without any server. Syncing directly between one's own devices, over a relay or through a server is a choice, not a requirement.
- **Sealed end to end.** What leaves a device is encrypted; servers and relays store and pass on what they cannot read.

DRAFT

- **The original stays the original.** Every document is kept byte for byte, named by its hash; a change makes a new version, never overwrites. Everything aktujo reads from it is a **suggestion** that a person confirms – nothing is changed behind their back.
- **Every step on record.** Who added, opened, changed, shared, approved or signed a document is written to an audit log that cannot be changed unnoticed.
- **One edition for everyone.** The same software for one person and for an enterprise; every feature free.
- **No telemetry.** aktujo connects only where it is needed or where its user asks it to.
- **Built to be built in.** The core is a set of libraries with a stable interface, so other applications can use it instead of writing their own.

4. The life of a document

Step	What aktujo does
Capture	Upload files and folders, import from a scanner or by mail; later the mobile app photographs paper directly.
Improve	Tools for scans and photos: find the page's edges, straighten a page shot at an angle (perspective), crop, rotate, black and white or greyscale, contrast; several pages into one document.
Read	The text layer of digital PDFs, OCR for scans. aktujo suggests the kind of document and its fields from rules that are data – kinds and rules a person defines for their own documents, whether letters, sheet music, research papers or a collection of books; templates to start from; programs that build aktujo in bring the rules of their trade.
Label	Tags, correspondents, kinds of documents and own fields; suggested from rules and from what was labelled before, confirmed by a person; rules that file new documents on their own once trusted.
Version	Every change – a new scan, an edited page, a signature – is a new version beside the old ones; any version can be opened, compared and restored.
Keep	Originals by hash and an event log of every change; encrypted on the device and on the way; retention and deletion by rule.
Find	Full-text search, also over encrypted documents on the device that holds the key; filters by kind, sender, date and field.
Share	With people and devices, with rights (read, add, manage); a household, a team, a tax adviser who may only read.
Workflows	Chains of steps across people: first person A approves, then person B signs, then person C countersigns or files – in order or side by side, with reminders, comments, rejections and a deadline per step; templates for recurring chains such as invoice approval, contracts or applications.
Sign	Check the signatures of documents that arrive; sign as a step of a workflow or on its own – also people without an account, by link; qualified signatures through the providers that are allowed to issue them (OPEN, see 7).
Audit	One log per document and per workspace: who did what and when, chained by hash and signed, so a gap or a change shows; exported as a report with the document.
Hand over	Pass a document on without copying it – an invoice becomes a receipt in spezado, same original, same hash; export everything at any time in an open format.

5. Where it runs

Setting	What it is
One computer	The app with its data in a folder and SQLite; no server, no account.
Own devices	The same, synced between laptop, desktop and phone – directly between the devices or through a relay that cannot read the data.

DRAFT

Setting	What it is
A server of one's own	A simple Docker Compose installation on a virtual machine or a small machine at home – or `aktujo serve` directly; the web app for every browser.
An organisation	Kubernetes for high availability: an operator that installs, updates and scales aktujo across several nodes, with PostgreSQL and S3-compatible object storage; login with OpenID Connect – for example through kernoid.

The data stays the same in every setting, so moving from one to the next is a sync, not a migration.

6. Built in, or on its own

aktujo's core is a set of Rust libraries: storage of originals (folder, S3, kernoid), the event log with sync, members and keys with end-to-end encryption, the relay; the document model, recognition and the scan tools. They are under the MIT licence, so any application may build them in – also apps published in app stores.

- **spezado** keeps its receipts in it and reads invoices with it.
- **notenwart** reads sheet music with its recognition.
- **kernoid** offers it to its accounts: as the place where aktujo's data lives in an organisation, and with aktujo's recognition for searching the files it stores.
- **aktujo itself** – command line, terminal, web app, later desktop and mobile apps – is the application anyone can use on its own, without any of the others.

Recognition is written in Rust so that it runs everywhere – on servers, on phones, in the browser – also over documents only the device can decrypt; established external programs (for example Tesseract) can be plugged in.

7. Security, audit and signatures

- **Encryption:** HPKE, XChaCha20-Poly1305 and Ed25519: every set of documents has its members, every member their devices and keys, every device what it needs.
- **Audit log:** every step is an event of the log the sync is built on – signed by the device that wrote it and chained by hash, so it is complete and cannot be changed unnoticed; in an organisation it can be kept longer than the documents it describes (OPEN: retention rules, GoBD).
- **Recovery** without a server that could read along: a recovery code, a second device, or – in an organisation – a recovery the organisation holds, if it chooses so.
- **Signatures** in three steps: first checking the signatures of incoming documents (PAdES); then signing within workflows – simple and advanced electronic signatures under eIDAS, with identities from an OpenID Connect provider such as kernoid; then qualified signatures through those who are allowed to issue them: qualified trust service providers and the European Digital Identity Wallet. aktujo does not become a trust service provider itself. **OPEN:** the legal requirements per kind of document, the state of the German wallet, Rust libraries for PAdES.

8. Licences and governance

aktujo is free software published by falkOSS – everything included, no telemetry. The program is under AGPL-3.0-or-later; the libraries other projects build on and the apps for the app stores are under MIT; this text is under CC BY 4.0.

9. The name

```
akto   a document that attests something
 -ujo  a container for it
     → aktujo
```

DRAFT

aktujo is Esperanto for a document folder. *akto* is an official document that attests something – a notice, a contract, a certificate – and the suffix *-ujo* makes a container for it. The Reta Vortaro, one of the standard dictionaries, describes an *aktujo* as a container for documents, at its simplest a folded sheet of cardboard [1]. Said ak-TU-jo, stressed on the second-to-last syllable like every Esperanto word.

10. Roadmap

1. The core: storage of originals, event log, encryption, local storage, relay.
2. Recognition in Rust, measured against Tesseract; the scan tools.
3. aktujo 0.1: one person, one computer – capture, improve, read, label, versions, keep (encrypted, in several spaces), find; also as aktujo-minimal, without scanning and OCR. Import from other programs comes later (OPEN: when).
4. Sync between own devices and sharing; the audit log; the mobile app with the camera.
5. Server and organisations: Docker Compose for one server; PostgreSQL, S3 and the Kubernetes operator for high availability; kernoid as login and storage.
6. Workflows and signatures: checking signatures, approval chains, signing, then qualified signatures.

Sources

[1] Reta Vortaro, article *akt/o*, entry *aktujo*. https://www.reta-vortaro.de/revo/art/akt.html (retrieved 2026-11-04).

DRAFT